

EVALUACIÓN	Obligatorio 1	GRUPO	TODOS	FECHA	08/2026
MATERIA	Desarrollo Full Stack integrado con IA				
CARRERA	Analista en Tecnologías de Información / Analista Programador				
CONDICIONES					
- Puntos: Máximo: 25 - Mínimo: 0 - Fecha máxima de entrega: 1/10/2026 hasta las 21:00 horas en gestion.ort.edu.uy.					
LA ENTREGA SE REALIZA EN FORMA ONLINE EN ARCHIVO NO MAYOR A 40MB EN FORMATO ZIP O RAR.					
Uso de material de apoyo y/o consulta					
Inteligencia Artificial Generativa					
• Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso. • Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó. • Verificar el contenido generado por la IA: No todo el contenido generado por la IA es correcto o preciso. Es esencial que los estudiantes verifiquen la información antes de usarla. • Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de "alucinaciones", los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso • En caso de existir dudas sobre la autoría, plagio o uso no atribuido de IAG, el docente tendrá la opción de convocar al equipo de obligatorio a una defensa específica e individual sobre el tema					
IMPORTANTE:					
1) Inscribirse 2) Formar grupos de hasta 2 personas del mismo dictado 3) Subir el trabajo a Gestión antes de la hora indicada (ver hoja al final del documento: "RECORDATORIO")					
Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails alamon@ort.edu.uy y rodriguez_mb@ort.edu.uy o telefónicamente al 29021505 - int 1156 u 1138					

Desarrollo de aplicación Fullstack

Objetivo general de la materia

El objetivo de la materia es crear una aplicación completa, con tecnologías modernas, arquitectura robusta, diseño visual profesional y que esté publicada.

Para este objetivo se trabajará sobre dos obligatorios a lo largo del semestre que evaluarán el desarrollo de backend (obligatorio 1) y desarrollo de frontend (obligatorio 2).

Si bien ambas entregas van a ser parte de una misma aplicación cada obligatorio evaluará aspectos concretos que **no serán reevaluados** más adelante. Si fuera

pertinente hacer ajustes al backend posteriores a la entrega del obligatorio 1, estos no serán evaluados en la segunda entrega.

La temática del obligatorio quedará a elección de los alumnos, debiendo tener en cuenta que en el tema elegido se contemplen los requerimientos funcionales y no funcionales que se plantean en **ambas letras de obligatorio**.

Las funcionalidades deberán ser implementadas siguiendo las buenas prácticas vistas en clase, será tan importante que la aplicación funcione, como la arquitectura utilizada al momento del desarrollo.

En este sentido se desaconseja la utilización de herramientas de IAG sin brindar contexto suficiente, ya que pueden brindar soluciones radicalmente diferentes a las vistas en clase. Debe haber un profundo entendimiento de todo el código generado.

Tener en cuenta que a lo largo de esta pauta se hablará del término genérico “documento”, refiriéndose a un documento almacenado en la base de datos, no hace referencia a ninguna temática específica.

Objetivo particular desarrollo Backend

Se deberá desarrollar un backend para la aplicación que deberá estar realizado con **nodeJS**.

La API deberá estar versionada en su versión 1, manejando el correcto ruteo que permita la escalabilidad de la aplicación y una futura implementación de nuevas versiones. Se deberá prestar también atención al correcto manejo de los status codes en las respuestas de las peticiones.

Todos los endpoints deberán tener las correspondientes respuestas de error esperables, por ejemplo: no debe haber dos usuarios con el mismo nombre de usuario, no se deben permitir campos vacíos. Si bien algunas de estas validaciones serán también responsabilidad del frontend es necesario duplicarlas para prevenir inyecciones indeseadas.

La temática de la API queda a elección del equipo de obligatorio, pero se deberán incluir las siguientes características:

Comentado [GS1]: valdría la pena aclarar acá que si aparecen cosas no vistas en clase las van a tener que justificar muy bien en la defensa y dejar claro que puede haber una pérdida de puntos considerables si no la pueden explicar?

Comentado [SF2R1]: Lo puse en la defensa, a ver si te parece. En ambas defensas lo puse

Requerimientos funcionales:

Rutas desprotegidas

- Registro de usuarios
 - La aplicación deberá permitir registrar usuarios. Solamente se deberán poder registrar usuarios con un perfil, pudiendo tener precargados en base de datos usuarios con otro tipo de perfil (ej: administradores del sistema)
- Login de usuarios

Comentado [GS3]: aclaramos que no son roles sino más bien "tipos" de usuarios que permiten hacer X cosa extra en la app?

Comentado [SF4R3]: Yo acá pensaba en dos roles, tipo normal y admin y que solo uno de ellos pueda cambiar de plus a premium. Subir la exigencia

Rutas protegidas

- Cambio de plan de usuario. Los usuarios que se registran podrán cambiar de plan plus (el que se asigna al momento del registro) a premium. Cambiar de plan solamente requiere estar en el plan plus. El administrador no necesita gestionar planes.
- Alta, baja, modificación, consulta y consulta con filtros de documentos de colecciones.
 - Las altas solamente se podrán hacer para 4 registros para los usuarios plus y van a ser ilimitadas para los usuarios premium. La entidad que maneja esta restricción queda a elección del equipo de desarrollo.
 - Las consultas deberán estar paginadas para colecciones que eventualmente podrían alojar mucha información.
- Alta, baja, modificación y consulta de documentos de categorías, que sirvan para asignar a las entidades del punto anterior. Por ejemplo, estos endpoints puede gestionar rubros de comidas (postres, entradas, pastas) en una aplicación de recetas de comidas. Esta podría ser una funcionalidad del administrador y deberá cuidar restricciones (ej: que no se pueda borrar una categoría que tenga documentos asociados).
- Subida de imágenes al servicio de alojamiento de imágenes de Vercel o a Cloudinary.
- Integración de inteligencia artificial generativa en un flujo. En caso de que el servicio devuelva indisponibilidad, esto no deberá condicionar el correcto funcionamiento de la aplicación. Esto no es un chat, implica que un endpoint internamente maneje alguna lógica de IAG.
- Investigación e implementación de algún endpoint que utilice algún recurso pertinente no visto en clase. Por ejemplo, conectarse con una API de terceros que brinde información pertinente al dominio, hacer un login con

Comentado [GS5]: es la intención ver paginación en este semestre?

Comentado [SF6R5]: Capaz que sea algo que investiguen e implementen? Me pareció una básica fundamental. Se puede sacar si se necesita

redes sociales, hacer envío de mails. Toda implementación de terceros debe funcionar en todo momento, asegurándose que no haya límites por uso.

Comentado [GS7]: habría que dar algun ejemplo?

Comentado [SF8R7]: Puede ser, se les ocurre alguno?

Comentado [SF9R7]: Ahí tiré algo

Documentación y tests de Postman

La documentación deberá estar compuesta por:

UNA COLECCIÓN

- La colección englobará todas las carpetas y es la que se exporta y se entrega.

CARPETAS

- Cada carpeta agrupará los endpoints que tengan relación entre sí.

REQUESTS

- Cada request corresponderá a un endpoint de la API
- Para cada request se deben generar los tests pertinentes para poder evaluar todas las funcionalidades desarrolladas en la API. Para cada endpoint se deben incluir todos los tests que permitan evaluar cada uno de los status codes de respuesta implementados.
A modo de ejemplo parte de un flujo esperado sería:
 - *Registro con datos incorrectos -> Registro exitoso (almacenamiento de las credenciales para siguientes requests) -> Registro erroneo por usuario duplicado.*
 - *Ingreso de 5 documentos en plan plus -> Error en el documento 5 por límite -> Cambio de plan al usuario -> Agrego documento 5.*
- Cada request debe testear la correcta devolución de datos, que el status code sea el esperado y todas las verificaciones que se entiendan necesarias.
- Ir asignando las variables de colección necesarias que necesiten los sucesivos endpoints.

VARIABLE DE COLECCIÓN

- Generar una variable de colección:
`prod_base_url` que apunte a la URL de base de la API publicada remotamente.
La documentación por defecto apuntará a la URL remota.

Comentado [GS10]: el año pasado no le encontré mucho sentido a esto. me parece que tiene más sentido que creen una sola variable que quede apuntando a prod, que igualmente me paso que al exportar la colección, pierde el valor de la var

Comentado [SF11R10]: Ahí lo ajusté un poco

Requerimientos no funcionales:

- La aplicación estará desarrollada con NodeJS, express, MongoDB, mongoose, JWT, Joi, BcryptJS, middleware para protección de rutas y todas las tecnologías y buenas prácticas aplicadas en el semestre.
- El deploy de la aplicación se hará en **Vercel**.
- La documentación y scripts de testeo se harán en **Postman**.

Se valorará:

- La correcta implementación de las buenas prácticas de OWASP top ten API para prevenir ataques (las vistas en clase).
- La atención a los detalles que aporten realismo a la aplicación.
- La concepción del entregable como un todo, no se espera solo código entregado, sino que también se respeten las arquitecturas que dan fácil escalabilidad y adaptabilidad al servidor y que se planifiquen y ejecuten los tests de los endpoints de manera profesional
- La implementación de funcionalidades innovadoras aparte de las solicitadas en la letra.

Cada docente establecerá un calendario de hitos de avance del obligatorio para poder hacer un seguimiento temprano del cumplimiento de los objetivos planteados.

PRODUCTO Y ESTRUCTURA DE LA ENTREGA DEL OBLIGATORIO

El producto final deberá ser una API Rest publicada.

Se deberá entregar una carpeta y dos archivos:

- Una carpeta con el código fuente de la API **EXCLUYENDO LA CARPETA NODE_MODULES** para no exceder los 40MB de peso y para facilitar la descarga de las entregas (aunque no se excedieran los 40MB).
- Archivo con un enlace a la URL de acceso a la API publicada.
- Archivo con la documentación exportada de Postman en formato JSON incluyendo la colección organizada en carpetas con los scripts de tests solicitados listos para correr.

DEFENSA

La defensa del obligatorio se realizará **entre el 05 y el 08 de octubre inclusive en modalidad ONLINE PARA TODOS LOS GRUPOS**. La misma será detallada por el docente. Esos días se dedican exclusivamente a defensas por lo que el grupo de obligatorio tendrá que asistir (online) solamente el día que tenga asignada la defensa. **Es una instancia OBLIGATORIA**. La no presentación implica la pérdida total de los puntos del obligatorio. En la defensa, como instancia de confirmación de autoría, se espera que haya un profundo conocimiento de todo el código generado, por lo que, no poder explicar en profundidad el código desarrollado será uno de los elementos considerados para quita de puntos.

IMPORTANTE:

Los requerimientos de la aplicación pueden sufrir ajustes durante el periodo de desarrollo que serán informados oportunamente por el docente.

Es responsabilidad de los integrantes del equipo de obligatorio tener en cuenta las aclaraciones o solicitudes sobre el trabajo realizadas por los docentes. Estas aclaraciones o solicitudes se consideran parte integral de la letra.

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

- **Obligatorios**

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la entrega online de obligatorios son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem "Evaluaciones" y la instancia de evaluación correspondiente, que figura bajo el título "Inscripto".
3. Para iniciar la entrega hacé clic en el ícono: 
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en "Agregar". El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en "Crear equipo". Cualquier integrante podrá:
 - Modificar la integración del equipo.
 - Subir el archivo de la entrega.
5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en "Subir" para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar un único archivo en formato zip o rar (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener un tamaño máximo de 40mb. Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.
6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La hora tope para subir el archivo será las 21:00 del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails, alamon@ort.edu.uy o rodriguez_mb@ort.edu.uy; o telefónicamente al 29021505 - int 1156 u 1138